Formal Methods for Critical Systems 2025/2026
MEIC @ FEUP

Second Project

Deadline: June 15, 2026, 20h

1 Introduction

The goals of this project are to practice the development of verified software systems, in particular using Dafny, Dafny's various language constructs, and some custom extensions to Dafny in order to implement some file-manipulating programs.

For this assignment, we provide boilerplate code as well as sample files. They are available from the course's website in Moodle.

2 Background

Dafny can be extended by binding trusted Dafny interfaces to $C\#$ code. For example, by default, Dafny does not support any command-line arguments. However, we can extend it to do so as shown in the `Io.dfy` and `IoNative.cs` files provided. Note that the class `HostConstants` has methods to determine how many command-line arguments were provided, as well as to retrieve them. It also has a function that lets you refer to those arguments in specification contexts. Note that each one is marked as `{:extern}`, which means both that the interface is axiomatically trusted and that Dafny will expect us to provide a $C\#$ implementation of each executable method. This is exactly what `IoNative.cs` provides. Notice that the names used in `IoNative.cs` carefully line up with those chosen in `Io.dfy` (though this isn't necessary if you specify the names when providing the extern annotation). To connect the two, pass `IoNative.cs` as an extra command-line argument to Dafny when compiling.

To make use of the IO routines, you can use Dafny's include mechanism. In another file, just write `include "Io.dfy"` at the top-level of the file, and include `IoNative.cs` on your command line when invoking Dafny. For example, you can execute the following command:

```
dafny yourFile.dfy IoNative.cs
```

You can also use the provided Makefile. **Download the zip file** `mfs_project2.zip`.

3 Challenge 1: Verified File Reverse Utility

Write a specification for `Main` that defines a line reverse utility. That is, the program expects two file names as command-line arguments, source and destination, and if the destination file doesn't already exist, it copies source to destination with all the lines in reverse order.

For example, suppose that an existing file named `SourceFile` contains

```
Line 1
Line 2
Line 3
Line 4
```

Then, if a file named `DestFile` doesn't exist and if we run the command[1]

```
./reverse SourceFile DestFile
```

we obtain a new file called `DestFile` that contains:

```
Line 4
Line 3
Line 2
Line 1
```

Write an implementation and prove that it meets your specification. Put your solution in a file called `reverse.dfy` and include it, along with any other files you depend on, in the `reverse` folder.

4 Challenge 2: Verified Grep Utility

The goal is to implement a verified version of the `grep` utility. Your program should run from the command line as:

```
./grep Word File
```

Your program should print

```
YES: n
```

if the word `Word` occurs in the file `File` with its first occurrence at position `n`. Otherwise, it should print

```
NO
```

At the core of this utility is a string matching algorithm. You should start with the naive solution (which runs in quadratic time). However, to obtain full marks, you will have to implement and verify

[1] We assume that after compilation, Dafny creates a file called `reverse` that can be executed. If not, adjust the command given to your own particular setup. For example, if it only creates a `DLL`, it can be executed as: `dotnet reverse.dll SourceFile DestFile`

the Knuth-Morris-Pratt algorithm [2]. For a textbook presentation of the algorithm, you might want to have a look at Chapter 32 of Cormen et al.'s Introduction to Algorithms [1].

Note that you will get partial marks if you only write correct functional specifications, even if you don't manage to implement the algorithms correctly. You must comment your code, so that we can understand your design decisions.

Put your solution in a file called `grep.dfy` and include it, along with any other files you depend on, in the corresponding folder (`grep-naive` for the naive version; `grep-kmp` for the Knuth-Morris-Pratt algorithm).

> **Bonus Points:**
>
> Bonus points will be given if your output is similar to the default output of the UNIX grep utility: instead of YES/NO output, it prints matching lines.
>
> You should also write a short report that explains the algorithms used and decisions.

5 Hand-in Instructions

The project is due on the **June 15, 2026, 20h**. You should follow the following steps to hand-in the project.

Preparing submission:

- Make sure your project is named `DafnyGXX.zip`, where `XX` is the group number. Always use two digits, e.g., Group 8's project should be named `DafnyG08`.
- `DafnyGXX.zip` is a zip file containing the solution and a README file where all group members are identified.

Upload the files in Moodle, in the link provided, before the deadline.

6 Project Evaluation

6.1 Evaluation components

In the evaluation of this project we will consider the following components:

1. Challenge 1: Correct implementation and specification of the requirements: 0–8 points.
   - Verified algorithm for reversing lines: 0–4 points
   - Copy of reversed source file to destination file: 0–4 points
2. Challenge 2: Correct implementation and specification of the requirements: 0–7 points

- Verified Naive solution: 2
- Verified Knuth-Morris-Pratt algorithm: 5

3. Quality, completeness, and simplicity of the obtained solution, plus report. (e.g. appropriate use of predicates): 0–4 points.
4. Presentation/defense of the work: 0–1 points.
   Please note: the presentation/defense may decrease the total mark for the project, depending on each individual student's performance.

Ensure that your specifications are as strong and precise as you can, given the interface defined in `Io.dfy`.

If any of the above items is only partially developed, the grade will be given accordingly.

After submission, students will present/defend their work.

The final grade for the project will be individual and the one after considering the work and presentation/defense.

6.2 Fraud Detection and Plagiarism

The submission of the project assumes the **commitment of honour** that the project was solely executed by the members of the group that are referenced in the files/documents submitted for evaluation. Failure to stand up to this commitment, i.e., the appropriation of work done by other groups or someone else, or sharing the work, either voluntarily or involuntarily, will have as consequence the immediate failure of this year's course of all students involved (including those who facilitated the occurrence).

6.3 Tips to write better specifications

- Know your Boolean operators!
  - If the method must satisfy different post-conditions based on the input value, use an implication to describe those conditional behaviors

    $$ensures\ P ==> Q1$$

    $$ensures\ !P ==> Q2$$

    The condition P should be based on values before the execution (and therefore use old), otherwise these assertions might not mean what you think they do
  - Avoid writing $b == true$ and $b == false$ when testing Boolean values, prefer b and $!b$ instead.
  - If a condition must be satisfied if and only if another condition is true (for example "the method returns true when the element has been added to the data structure") use an equivalence $<==>$ rather than multiple implications.
- Keep it small and simple

– Keeping things simple is useful when coding, but even more important for specification. Your specification should be complete, but if it is complex and unreadable it is not likely to be useful (or correct).

GOOD LUCK!

References

[1] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2009.

[2] Donald E Knuth, James H Morris, Jr, and Vaughan R Pratt. Fast pattern matching in strings. *SIAM journal on computing*, 6(2):323–350, 1977.